

Incident Response Playbook

NIST 800-61r2 program +
mini-SIEM triage

AT A GLANCE

63

Auth+web events

10

Alerts raised

6

Playbooks

16 pages

IR PDF

63

Timeline rows

8 / 8

Tests re-run

NIST 800-61r2

Standard

VERDICT

TESTS 8/8

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

EXISTING PDF KEPT

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

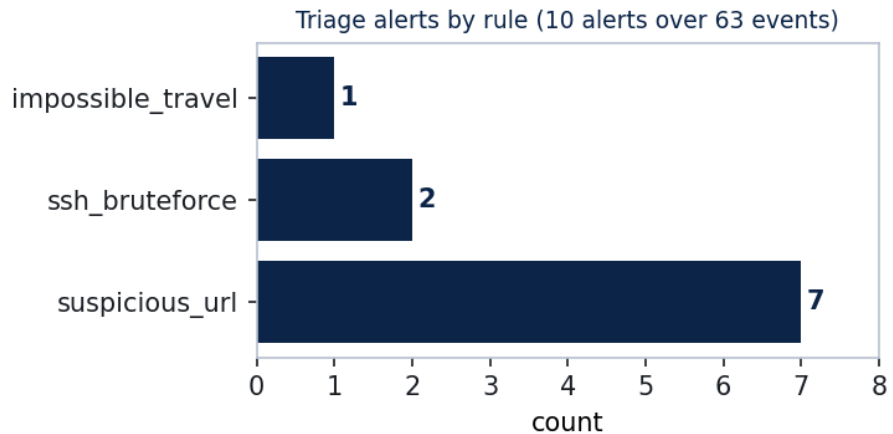
Objective & brief mapping

Two things that fit together: a complete Incident Response program aligned to NIST SP 800-61r2 (an IR plan, six scenario playbooks, a report template, delivered as markdown and a navy PDF), and a SIEM-style log-triage tool that parses synthetic auth/web logs, fires detection rules, and produces an ordered incident timeline.

Architecture

generate_logs.py writes declared-synthetic auth.log + access.log. log_triage.py parses and runs three rules into alerts.json + incident_timeline.csv. build_docs.py renders the plan and playbooks; build_pdf.py renders the 16-page IR PDF into results/evidence.

```
generate_logs -> auth.log + access.log (SYNTHETIC)
log_triage -> rules -> alerts.json + timeline.csv
build_docs + build_pdf -> IR_Plan_and_Playbooks.pdf (16 pp)
```



Triage alerts by rule, charted from alerts.json.

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # logs -> triage -> docs -> PDF (~0.4 s)
python3 unittest-style runner over tests/ # 8 tests
```

The artefacts each figure and table above are drawn from:

- **alerts.json** — 10 triage alerts
- **incident_timeline.csv** — 63 ordered rows
- **evidence/IR_Plan_and_Playbooks.pdf** — 16-page IR program (kept as-is)
- **docs/alerts_by_rule.png** — chart from alerts.json

Method & implementation

Three rules run over parsed events: ssh_bruteforce (≥ 5 failures/IP in 60 s, escalating to critical if a success follows the burst = likely takeover, MITRE T1110); impossible_travel (one user from two IPs within an hour, T1078); suspicious_url (SQLi, traversal, XSS, webshell, admin-probe patterns). The same logic is shown as Splunk SPL and Sentinel KQL for a real SIEM.

Results

From results/alerts.json: 26 auth events + 37 web events parsed, 10 alerts raised.

Rule	Count	Notable
ssh_bruteforce	2	198.51.100.23 CRITICAL (12 fails then success)
impossible_travel	1	carol from two IPs 6 min apart
suspicious_url	7	SQLi, /etc/passwd, webshell, XSS, admin
Timeline	63 rows	time-ordered incident_timeline.csv

Detection rules -> MITRE

Rule	Condition	MITRE
ssh_bruteforce	≥ 5 fails/IP/60 s	T1110
(escalation)	success after burst = takeover	T1078
impossible_travel	2 IPs/user/1 h	T1078
suspicious_url	SQLi/traversal/XSS/webshell	T1190/T1083/T1505.003

Verification

Re-ran here, 8/8 (~0.1 s). The triage engine and doc/PDF builders are pure-Python, so every test executed as recorded.

Incident Response Playbook

NIST 800-61r2 program + mini-SIEM triage

AT A GLANCE

63
Auth+web events

10
Alerts raised

6
Playbooks

16 pages
IR PDF

63
Timeline rows

8 / 8
Tests re-run

NIST 800-61r2
Standard

VERDICT

TESTS 8/8

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

EXISTING PDF KEPT

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Test group	Checks	Result
Parsing	auth + web events parsed	PASS
Rules	brute-force escalation, impossible travel	PASS
Suspicious URL	injection/traversal/XSS matched	PASS
Docs/PDF	plan + 6 playbooks + PDF built	PASS
Total	8 tests	8/8

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

The Python rules mirror what a real SIEM would run; I ship the Splunk SPL and Sentinel KQL equivalents.

Ran here	Real SIEM	Rule
ssh_bruteforce	Splunk SPL	count>=5 fails/IP in 1 min
impossible_travel	Sentinel KQL	two IPs per user in 1 h
suspicious_url	ELK detection rule	SQLi/traversal/XSS regex

Ethics, data integrity & declared substitutions

Defensive documentation and teaching-scale triage. **Data integrity:** all logs are synthetic and declared — no real user, credential or host; IPs are RFC 5737 documentation ranges. **Existing deliverable kept:** the project already ships results/evidence/IR_Plan_and_Playbooks.pdf (16 pages), which I left untouched and reference here; this report is the standard docs/09_Report.pdf added alongside it. "Impossible travel" is a two-IP proxy, not a true geo-velocity calc.

Learnings

Connecting the triage output to the playbooks closed the loop for me: an alert is only useful if a responder knows exactly which runbook it opens. Escalating brute-force to critical *only* when a success follows the burst is the kind of context rule that separates noise from a real account-takeover signal.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The triage engine is teaching-scale: single-line regex parsing, fixed thresholds, no enrichment.
- "Impossible travel" is a two-IP proxy, not a true geo-velocity calculation.
- The PDF builder implements a small Markdown subset, not full CommonMark.
- Playbooks are a starting template; a real program tailors roles and legal steps.
- Aligned to NIST SP 800-61r2 five phases; six scenario playbooks shipped.
- ssh_bruteforce escalates to critical when a success follows the failure burst.
- All logs are synthetic; IPs are RFC 5737 documentation ranges.
- MITRE mappings: T1110, T1078, T1190/T1083/T1505.003.
- Parsed 26 auth events + 37 web events; raised 10 alerts.
- ssh_bruteforce: 198.51.100.23 escalated to CRITICAL (12 fails then success).
- impossible_travel: user carol from two IPs 6 min apart.
- suspicious_url: 7 hits incl. /etc/passwd and a webshell upload.
- incident_timeline.csv has 63 time-ordered rows.
- The IR PDF is 16 pages (~34 KB), kept untouched in results/evidence.
- Splunk SPL and Sentinel KQL equivalents shipped in the README.
- Aligned to NIST SP 800-61r2; six scenario playbooks.
- Reference: NIST SP 800-61r2; MITRE ATT&CK; SANS IH Handbook.
- Reference: Splunk Search Reference; Elastic detection-rules.
- Reference: CISA incident-response playbooks.
- alerts.json records 10 alerts across the three rules.
- The 16-page IR PDF in results/evidence was kept untouched.